

Quantization-Aware Compression of Vision-Language Models for Efficient Edge Deployment

Computer Vision Final Project Proposal

1. Problem Statement

Vision-language models such as CLIP (Contrastive Language-Image Pretraining) have become foundational tools in computer vision, enabling tasks like zero-shot image classification, image-text retrieval, and open-vocabulary detection — all without task-specific training. However, these models are large: CLIP ViT-B/32 contains over 150 million parameters and requires significant memory and compute at inference time, making deployment on edge hardware — mobile chips, NPUs, and embedded accelerators — impractical in its current form.

Quantization is one of the most widely used techniques for model compression: it replaces the high-precision floating-point numbers the model uses internally (32-bit) with lower-precision integers (8-bit or 4-bit), reducing model size and inference latency with minimal retraining. While quantization is well-studied for language models and standard image classifiers, its effect on vision-language models is less understood. CLIP's joint image-text embedding space is sensitive — small perturbations in one modality can misalign representations in the other — making it non-obvious how aggressively it can be compressed without breaking its core capabilities.

This project is interesting for two reasons. First, it addresses a real deployment bottleneck: every major chip company (Qualcomm, Intel, AMD) ships inference hardware optimized for INT8 workloads, yet the most capable vision-language models do not run efficiently on them out of the box. Second, the sensitivity analysis question — which parts of a vision-language model tolerate quantization and which do not — is a genuine open research question with direct engineering implications.

2. Background Reading

The following works provide the necessary context for this project:

- [1] Radford et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. ICML 2021. — The original CLIP paper; defines the model architecture, training objective, and zero-shot evaluation protocol.
- [2] Jacob et al. (2018). Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. CVPR 2018. — Foundational work on quantization-aware training (QAT) and the fake-quantization training framework.
- [3] Nagel et al. (2021). A White Paper on Neural Network Quantization. arXiv:2106.08295. — Comprehensive survey of post-training quantization (PTQ) methods, calibration strategies, and

per-channel vs. per-tensor quantization.

- [4] Xiao et al. (2023). SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. ICML 2023. — Introduces migration of quantization difficulty from activations to weights; directly applicable to transformer-based vision models.
- [5] Yao et al. (2022). ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers. NeurIPS 2022. — Practical PTQ pipeline for transformer architectures with layer-wise sensitivity analysis.
- [6] Lin et al. (2023). AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv:2306.00978. — Identifies salient weights using activation statistics; applicable to CLIP's attention and projection layers.

3. Data

This project uses two standard, publicly available benchmark datasets. No new data collection is required.

CIFAR-100 is used for zero-shot image classification. It contains 60,000 images across 100 object categories and is available directly through PyTorch's torchvision library. CLIP is evaluated on CIFAR-100 by matching each image against text prompts of the form "a photo of a {class}" — no task-specific fine-tuning is needed. CIFAR-100 is small enough for rapid iteration and well-suited for measuring how much zero-shot classification accuracy degrades under quantization.

Flickr30k is used for image-text retrieval. It contains approximately 31,000 images, each annotated with five human-written captions. The evaluation task is bidirectional retrieval: given a text query, retrieve the correct image from the full test pool (image retrieval), and vice versa (text retrieval). Flickr30k tests whether the alignment between CLIP's two modalities is preserved under compression — a more sensitive and realistic probe than classification alone. The dataset is freely available via the University of Illinois upon completing a short access form.

4. Method

The project is structured as a series of experiments, each building on the last. The goal is to progressively compress CLIP and measure what is lost at each step.

Stage 1 — Establish a baseline. First, the original unmodified CLIP model is run on both datasets to record how accurate it is and how fast it runs. These numbers serve as the reference point that all later experiments are compared against.

Stage 2 — Compress the model without retraining. The model's internal numbers are rounded down from high precision to lower precision — think of it like reducing an image from RAW format to JPEG. This is done in two steps: first to medium compression (8-bit), then to heavy compression (4-bit). After each step, the model is tested again to see how much accuracy was lost and how much faster it now runs. Existing open-source tools handle the

compression automatically; no model training is required at this stage.

Stage 3 — Find the model's weak points. Rather than compressing the whole model at once, each section of the model is compressed individually while leaving the rest untouched. This reveals which sections of the model are robust (accuracy stays the same when compressed) and which are fragile (accuracy drops sharply). The result is a map of the model's sensitivity to compression.

Stage 4 — Compress smarter using the sensitivity map. Using the findings from Stage 3, a mixed strategy is applied: robust sections are compressed heavily (4-bit) to save the most space, while fragile sections are compressed lightly (8-bit) or left untouched. This aims to get the best of both worlds — a small, fast model that still performs well.

Optional Stage 5 — Fine-tune the compressed model. If time permits, the compressed model is given a short additional training pass where it learns to compensate for the loss of precision. This typically recovers a small amount of the accuracy lost during compression.

All tools and pretrained models used in this project are freely available online. The project does not train any model from scratch.

5. Evaluation and Expected Results

Results will be reported as a tradeoff table and Pareto curve across the following metrics:

- **Accuracy:** Top-1 accuracy on CIFAR-100 zero-shot classification; Recall@1 and Recall@5 on Flickr30k image-text and text-image retrieval.
- **Model size:** Parameter storage in MB for each quantization configuration.
- **Inference latency:** Average milliseconds per image on CPU (targeting Intel OpenVINO or standard PyTorch CPU backend).
- **Sensitivity profile:** Per-layer accuracy drop under individual block quantization, visualized as a bar chart.

Expected results are as follows. Medium compression (8-bit) is expected to preserve accuracy on CIFAR-100 within 1-2% of the original while reducing model size by approximately 4x and inference speed by 2-3x. Heavy compression (4-bit) is expected to cause a larger accuracy drop of 3-6% but shrink the model by approximately 8x. The sensitivity analysis is expected to show that the early image-processing layers and the final output layers are most fragile, consistent with findings in the existing literature. The mixed-precision approach is expected to recover most of the accuracy lost under heavy compression while retaining most of the size reduction. The main deliverable is a clear characterization of the accuracy-efficiency tradeoff for CLIP under different compression strategies, with a practical recommendation for edge deployment.

